AI-Assistant Assignment-9.1

2303A52483

Batch-44

Problem 1:

Consider the following Python function:

def find_ max(numbers):

return max(numbers)

Prompt

Write a Python function that finds the maximum number from a list of numbers using the built-in max() function.
Document the function using three documentation styles: **docstring**, **inline comments**, and **Google-style documentation**. Then compare the three styles and explain which one is best for a mathematical utilities library.

Task:

• Write documentation for the function in all three formats:

(a) Docstring

(b) Inline comments

(c) Google-style documentation

• Critically compare the three approaches. Discuss the

advantages, disadvantages, and suitable use cases of each style.

• Recommend which documentation style is most effective

for a mathematical utilities library and justify your answer

```python
#Consider the following Python function:
#def find_max(numbers):
#return max(numbers)
#Task:
#• Write documentation for the function in all three formats:
#(a) Docstring
#(b) Inline comments
#(c) Google-style documentation
#(a) Docstring
def find_max(numbers):
    """
    This function takes a list of numbers as input and returns the maximum value from the list.

    Parameters:
    numbers (list): A list of numerical values.

    Returns:
    max_value: The maximum value found in the input list.
    """
    return max(numbers)
#(b) Inline comments
def find_max(numbers):
    # Use the built-in max function to find the maximum value in the list
    return max(numbers)
```

```python
#(b) Inline comments
def find_max(numbers):
    # Use the built-in max function to find the maximum value in the list
    return max(numbers)
#(c) Google-style documentation
def find_max(numbers):
    """
    Finds the maximum value in a list of numbers.

    Args:
        numbers (list): A list of numerical values.
    Returns:
        The maximum value found in the input list.
    """
    return max(numbers)
nums=[3, 7, 2, 9, 5]
print(find_max(nums))  # Output:
```

```
TERMINAL
PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe"
Assit/Assignment 9.1.py"
9
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation of the Program

1. The function **find_ max (numbers)** receives a list of numbers as input.

2. It uses Python's built-in **max ()** function to find the largest value in the list.
3. The same function is documented in three styles:
    - **Docstring** → General Python documentation.
    - **Inline comments** → Explains the logic inside the code.
    - **Google-style docstring** → Structured format commonly used in industry.

Execution

Input list:

Num= [3, 7, 2, 9, 5]

Step:

- The function checks all numbers in the list.
- The largest number in the list is **9**.

Output

9

<u>Problem 2: Consider the following Python function:</u>

def login (user, password, credentials):

return credentials. get (user) == password

Task:

1. Write documentation in all three formats.

2. Critically compare the approaches.

3. Recommend which style would be most helpful for new

developers onboarding a project, and justify your choice.

P**rompt:**

Write a Python function to check whether a user login is valid by comparing the given username and password with stored credentials in a dictionary. Document the function using docstring, inline comments, and Google-style documentation, then compare which style is best for new developers.

```python
#Consider the following Python function:
#def login(user, password, credentials):
#return credentials.get(user) == password
#Task:
#1. Write documentation in all three formats.
#2. Critically compare the approaches provide total code.
#3. Recommend which style would be most helpful for new
#developers onboarding a project, and justify your choice.
#1. Documentation in all three formats:
#(a) Docstring
def login(user, password, credentials):
    """
    This function checks if the provided user and password match the credentials stored in a dictionary.

    Parameters:
    user (str): The username to be authenticated.
    password (str): The password to be authenticated.
    credentials (dict): A dictionary containing username-password pairs.

    Returns:
    bool: True if the credentials match, False otherwise.
    """
    return credentials.get(user) == password
#(b) Inline comments
def login(user, password, credentials):
    # Check if the provided user exists in the credentials dictionary and if the password matches
    return credentials.get(user) == password
```

```python
#(c) Google-style documentation
def login(user, password, credentials):
    """
    Authenticates a user based on provided credentials.

    Args:
        user (str): The username to be authenticated.
        password (str): The password to be authenticated.
        credentials (dict): A dictionary containing username-password pairs.

    Returns:
        bool: True if the credentials match, False otherwise.
    """
    return credentials.get(user) == password
```

```
#2. Critical comparison of approaches:
#- Docstrings provide a comprehensive description of the function, including parameters and return values. They are easily accessible through help()
#- Inline comments are useful for explaining specific lines of code but can become cluttered if overused. They are not as structured as docstrings a
#- Google-style documentation is similar to docstrings but follows a specific format that can be easily parsed by documentation generators. It provi
#3. Recommendation: Google-style documentation is most helpful for new developers onboarding a project because it provides a consistent, structured
```

```
Docstring version: 15
Inline version: 15
Google version: 15
Docstring version: True
Inline version: True
Google version: True
```

Observation of the Program

1. The function **login(user, password, credentials)** is designed to **authenticate a user**.
2. It checks whether:
    - The username exists in the credentials dictionary.
    - The stored password matches the entered password.
3. The comparison is done using:
4. credentials.get(user) == password
    - credentials.get(user) returns the stored password if the user exists.
    - If the user does not exist, it returns None, which results in False.

Documentation Observation

The same function is documented using three styles:

- **Docstring:** Gives a general explanation of the function, parameters, and return type.
- **Inline comments:** Explain the logic directly near the code line.
- **Google-style docstring:** Provides structured, readable, and tool-friendly documentation.

Functional Behaviour Observation

- If the username and password match → Function returns **True** (login successful).
- If they do not match or user not found → Function returns **False** (login failed).

Final Lab Observation Statement

The program successfully demonstrates three documentation styles for a user authentication function. The function correctly verifies login credentials using a dictionary lookup. Among the documentation approaches, Google-style documentation is the most structured and helpful for onboarding new developers because it clearly describes inputs, outputs, and purpose in a consistent format.

Task 3:

Prompt:

#Create a Python module named calculator.py and show how to generate automatic documentation.

#Requirements:

#Write the file calculator.py with these functions using proper docstrings:

Add (a, b) → return sum

Subtract (a, b) → return difference

Multiply (a, b) → return product

#divide (a, b) → return quotient (handle division by zero)

#Show the terminal commands to display module documentation using Python documentation tools.

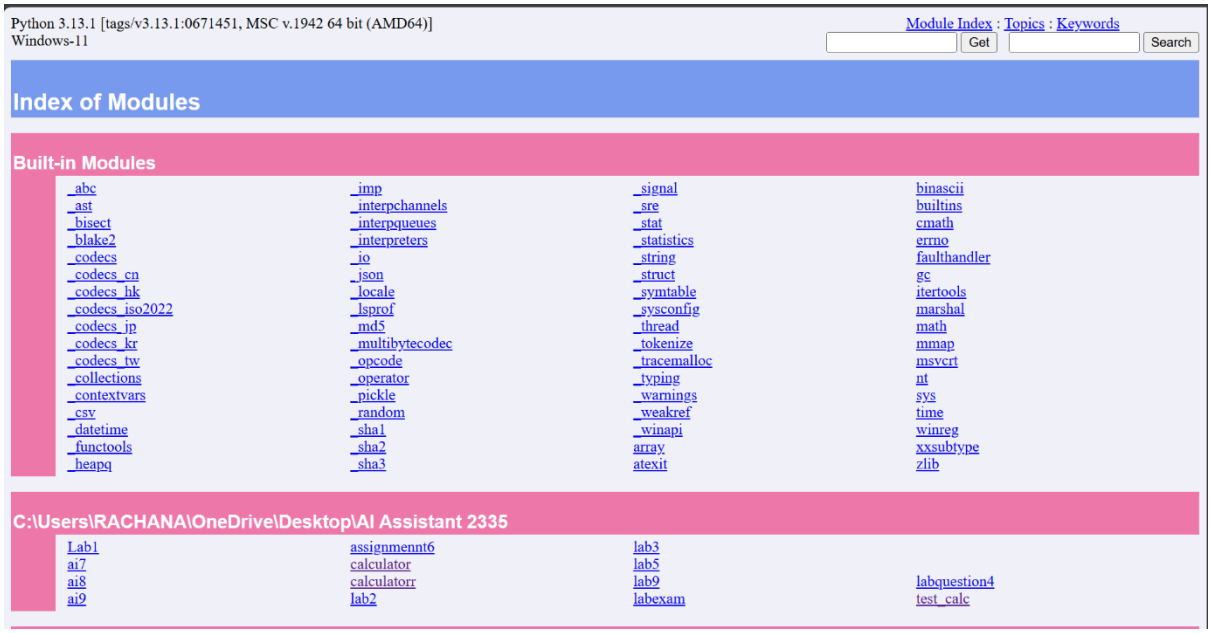
#Show the commands to generate HTML documentation using pydoc.

#Show how to open the generated HTML file in a web browser.

#calculator.py

```python
#Create a Python module named calculator.py and show how to generate automatic documentation.
#Requirements:
#Write the file calculator.py with these functions using proper docstrings:
#add(a, b) → return sum
#subtract(a, b) → return difference
#multiply(a, b) → return product
#divide(a, b) → return quotient (handle division by zero)
#Show the terminal commands to display module documentation using Python documentation tools.
#how the commands to generate HTML documentation using pydoc.
#Show how to open the generated HTML file in a web browser.
#calculator.py
def add(a, b):
    """
    Returns the sum of a and b.

    Parameters:
    a (float): The first number.
    b (float): The second number.

    Returns:
    float: The sum of a and b.
    """
    return a + b
```

```python
def subtract(a, b):
    """Returns the difference of a and b.

    Parameters:
    a (float): The first number.
    b (float): The second number.

    Returns:
    float: The difference of a and b.
    """
    return a - b
def multiply(a, b):
    """Returns the product of a and b.

    Parameters:
    a (float): The first number.
    b (float): The second number.

    Returns:
    float: The product of a and b.
    """
    return a * b
def divide(a, b):
    """Returns the quotient of a and b. Handles division by zero.

    Parameters:
    a (float): The first number.
    b (float): The second number.

    Returns:
    float: The quotient of a and b, or None if division by zero occurs.
    """
    if b == 0:
        print("Error: Division by zero is not allowed.")
        return None
    return a / b
```



Result:

The calculator.py module was created with proper docstrings. Using the tool, automatic documentation was generated and displayed in a web browser. All arithmetic functions (add, subtract, multiply, divide) appeared correctly in the gen

Task 4:

Prompt:

Create a Python module named conversion.py.

Requirements:

1. The module should contain functions:

   - decimal_ to_ binary(n)

   - binary_ to_ decimal(b)

   - decimal_ to_ hexadecimal(n)

2. Each function must include clear Google-style docstrings.

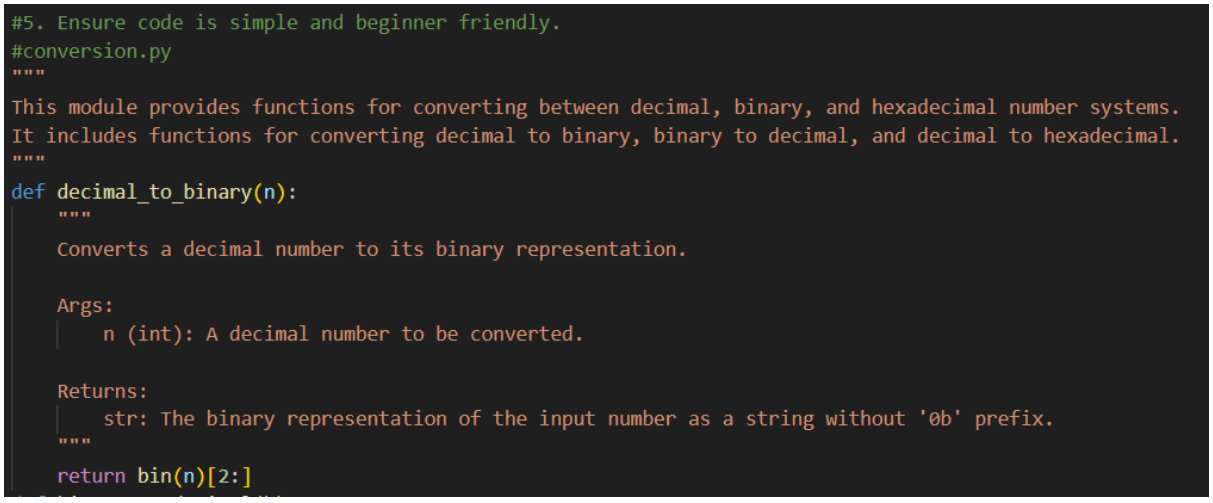
   - decimal_ to_ binary(n): return binary string without '0b'

   - binary_ to_ decimal(b): accept string and return integer

   - decimal_ to _ hexadecimal (n): return hexadecimal string without '0x'

4. Add a module level docstring describing the purpose of the module.

5. Ensure code is simple and beginner friendly.



```python
#5. Ensure code is simple and beginner friendly.
#conversion.py
"""
This module provides functions for converting between decimal, binary, and hexadecimal number systems.
It includes functions for converting decimal to binary, binary to decimal, and decimal to hexadecimal.
"""
def decimal_to_binary(n):
    """
    Converts a decimal number to its binary representation.

    Args:
        n (int): A decimal number to be converted.

    Returns:
        str: The binary representation of the input number as a string without '0b' prefix.
    """
    return bin(n)[2:]
```

```python
def binary_to_decimal(b):
    """Converts a binary string to its decimal representation.

    Args:
        b (str): A binary string representation of a number.

    Returns:
        int: The decimal representation of the input binary string.
    """
    return int(b, 2)
def decimal_to_hexadecimal(n):
    """Converts a decimal number to its hexadecimal representation.

    Args:
        n (int): A decimal number to be converted.

    Returns:
        str: The hexadecimal representation of the input number as a string without '0x' prefix.
    """
    return hex(n)[2:]
# Sample function calls
print(decimal_to_binary(10))  # Output: '1010'
print(binary_to_decimal('1010'))  # Output: 10
print(decimal_to_hexadecimal(255))  # Output: 'ff'
```

```
 Assit/Assignment 9.1.py"
9
1010
10
ff
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users
 Assit/Assignment 9.1.py"
9
1010
10
ff
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```